Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

Digitális Egyéni Tanulási Platform

Készítette: Buczkó Béla
Kovács Péter

2026.

Tartalom

Projektötlet - Digitális Egyéni Tanulási Platform ..... - 4 -
1. A rendszer fő elemei ..... - 4 -
2. Fejlesztési megfontolások ..... - 8 -
Projekt megvalósítása ..... - 10 -
Felhasználói Dokumentáció ..... - 12 -
Bejelentkezés/regisztráció ..... - 12 -
Tanulói Oldal ..... - 12 -
Tanulói oldalsáv: ..... - 13 -
Tanulói főoldal ..... - 13 -
Tanuló kurzusai ..... - 14 -
Kurzus fal ..... - 14 -
Üzenetek ..... - 15 -
Kereső ..... - 15 -
Értesítések ..... - 16 -
Pénzügyek ..... - 17 -
Beállítások ..... - 17 -
Tanári Oldal ..... - 17 -
Tanári oldalsáv ..... - 18 -
Tanári főoldal ..... - 18 -
Kurzusaim ..... - 19 -
Tanulóim ..... - 20 -
Üzenetek ..... - 22 -
Órarendem ..... - 22 -
Értesítések ..... - 23 -
Pénzügyek ..... - 23 -
Beállítások ..... - 23 -
Frontend Felhasználási útmutató ..... - 24 -
Célunk: ..... - 24 -
Felhasználói folyamat: ..... - 24 -
Software Architecture Document ..... - 26 -
1. Tech-Stack ..... - 26 -
2. C4 Diagram ..... - 26 -
- Modules implemented in the prototype ..... - 28 -
- Modules planned for future development ..... - 29 -
3. Database Schema ..... - 29 -

Identity and Cities Module ..... - 30 -
CourseBase module ..... - 31 -
Tutoring module ..... - 31 -
Engagement module ..... - 32 -
Scheduling Module ..... - 32 -
Payment Module ..... - 32 -
Resources Module ..... - 33 -
LearningPath Template Module ..... - 34 -
Learning Path Progression Module ..... - 34 -
4. API endpoint documentation ..... - 35 -
Használt források ..... - 38 -
Frontend ..... - 38 -
Backend ..... - 38 -
Adatbázis ..... - 38 -
Infrastruktúra ..... - 38 -
Tervezési eszközök és egyéb ..... - 38 -

Projektötlet - Digitális Egyéni Tanulási Platform

A projekt lényege egy központosított, kurzusokon alapuló magántanári platform létrehozása, melyre a felregisztráló tanárok állíthatják össze kurzusaikat, akár a hagyományos óratartásos módszerrel, vagy tanulói ösvények kialakításával, és a tanulni vágyók, vagy azok szülei ezek közül kereshetik ki ezek közül a számukra legjobbat.

Alapja, hogy a tanulók egy rugalmas, személyre szabott és közösségi tanulási rendszerhez férjenek hozzá, amely egyszerre integrálja:

- hagyományos óratartásos rendszert, tanulói fallal
- a tanulói ösvényeken alapuló kurzusokat
- a videós leckéket
- az egyéb formátumú tanagyagokat (szöveg, prezentáció, fájl stb.), tanári fájlkezelőt
- a beadható feladatokat, feladatlapokat
- a konzultációs/online órákat, időpontfoglaló rendszerrel
- a közösségi funkciókat, mint nyilvános chat fal, és közlemény fal
- közösségen belüli webináriumok tartásának lehetőségét
- az AI-alapú segítő funkciókat
- számlázási nyilvántartó funkciókat

---

1. A rendszer fő elemei

1. **Kurzuskereső rendszer**
   A felregisztráló tanárok saját kurzusokat adhatnak a profiljaikhoz, melyek között a tanulók maguk válogathatnak. A kurzusok összeállítása felett a tanár teljes kontrollal rendelkezik, megad:

   - szűrésre alkalmas adatokat (pl. tárgy vagy inkább tag-ek, nehézségi szint, témák)
   - leírást
   - kurzus típusát
     - hagyományos 1:1 tanítási mód
     - tanulói ösvény
   - irányár (konkrét, vagy határok)

   Amennyiben tanulói ösvényt választott, továbbá megad:

   - kisebb modulokat, amik egy-egy témát (akár altémát), részt dolgoznak fel

    - tartalmazhatnak videóleckét, szöveges részt, fájlokat, prezentációkat, feladatlapokat, tanulói kártyákat
- általános tanulói útvonala(ka)t a kisebb modulokból (több, ha esetleg lennének szintek külön)

A tanulói útvonalat aztán a tanár vagy a rendszer a tanulóra szabhatja, akár teljesen újat készíthet.

---

2. **Egyedi tanulói ösvény**

A tanuló a kurzus felvétele után:

- kiválaszthatja mennyit szeretne tanulni hetente
- a rendszer ehhez rendel egy maximum konzultáció számot
- annyi modul nyílik meg amennyit vállalt
- ezek teljesítése kötelező
- választása szerint csatlakozhat a kurzus közösségéhez, ahol szöveges csatornákon cseveghet a kurzus többi résztvevőjével
- a hirdetőfalon a tanár eseményei (pl. webináriumok) és egyéb kiírásai találhatóak

Pénzügyi követés:

- csak akkor nyílnak meg új modulok, ha be lett fizetve a tandíj
- a diáknak jeleznie kell, hogy befizette
- a tanár pedig felülvizsgálja, igaz-e, az ő visszaigazolása alapján nyílik meg a haladás

---

3. **Hagyományos magánórák**

Amennyiben a tanár (vagy a diák) inkább a személyesebb 1:1 tanítási módszert preferálja, a platform minden eszközt megad hozzá.

A tanuló a kurzus felvételénél:

- kiválasztja heti hány órában szeretne tanulni, mi a célja
- a tanár által szabadnak jelölt időpontok közül kiválaszt egyet az első konzultációra

A konzultáció után, ha mindenki elégedett volt a tanár elfogadhatja a jelentkezést, és innentől elérhetővé válnak:

- heti órafoglalások maximum a kiválasztott óraszámban
- tanulói fal, ahol a tanár kiadhatja az új anyagokat, feladatokat, dolgozatokat, házikat
- külön beadandók nézet
- haladási mutató és adott témakör melyeket a tanár ad meg a célhoz mérten

Pénzügyi követés:

- csak akkor érhetőek el az órai anyagok/új kiadott feladatok, ha be lett fizetve a tandíj
- óránként számolja a rendszer, de egyszerre több befizetése is lehetséges
- a diáknak jeleznie kell, hogy befizette
- a tanár pedig felülvizsgálja, igaz-e, az ő visszaigazolása alapján nyílik meg a tanulói fal új része

---

4. **AI-támogató rendszer:**
   Az AI mint asszisztens jelenik meg a projektben. Célja a tanárok munkájának, és a diákok könnyebb haladásának segítése.

   Tanári funkciók:

   - chatbot:
     - kontextusként kijelölhető diák profil
     - fájlok, anyagok, feltöltött szöveg
     - segíthet tananyag személyre szabásában, átalakításában, egyéni tanmenet kialakításában
     - esetleg ötletelésben
   - modul/anyag, feladatlap, tanuló kártya készítő:
     - kontextusként megadható fájlok, anyagok, feltöltött szöveg
     - azok alapján generál egy új, a kiválasztott formának megfelelő tartalmat
     - a tanár felülvizsgálhatja, átírhatja
   - beadott feladat előértékelése, visszajelzés generálása
   - ügyfélszolgálati chatbot
   - közösségi moderálás

   Tanulói funkciók:

   - chatbot:
     - tanulást segítő funkció
     - megértés segítése, átfogalmazás

---

5. **Konzultációs/online óra rendszer**
   A tanuló a vállaltaknak megfelelő óraszámban foglalhat hetente konzultációt/órát a tanárnál.

   - a tanár állítja be, milyen időpontokban foglalható
   - tanár és tanuló részről is Google/Outlook Calendar integráció
   - a konzultációk hosszát a tanár adja meg
   - az online órák/konzultációk időpontjának halasztását, áttételét lehet kérvényezni mindkét oldalról, új javaslattétellel is, amit a másik fél elfogadhat vagy elutasíthat, de lemondásra is sor kerülhet

6. **Közösségi élmény**

A platform saját közösségi funkciókat kínálna, melyeket a tanár szabhat személyre:

- egy hirdetőfal, ahová a kurzus készítője írhat bejelentéseket
- továbbá megjelennek a kurzusközösség eseményei
- események: webinárok, extra órák, workshopok
- a tanár tetszése szerint személyre szabhat, létrehozhat szöveges csatornákat
- ezeken a kurzus résztvevői tudnak egymással is kommunikálni, segíteni egymást

---

7. **Egyedi tanulást segítő funkció**

Hogy minél jobban a tanuló igényeihez és képességeihez igazítható legyen a haladása, regisztrációnál sor kerülne egy rövid „tanulói jellem" tesztre, ami alapján a rendszer megállapítja a tanuló tanulási stílusát, milyen módszerek vállnak be jobban, mi megy kevésbé jól, mi motiválja stb. Ez a tanuló profiljába kerül mentésre, amit a tanár és a segítő AI is láthat.

A kurzus felvételénél pedig akár a tanár állíthat össze szintfelmérőt, ami alapján elhelyezhető a tanuló jelenlegi tudása, és megadhatja a célját, mit szeretne elérni.

---

8. **Pénzügyi átláthatóság**

Hogy könnyen nyomon követhetővé váljon a tanuló és a tanár számára is a befizetések és elmaradások nyilvántartása, a rendszer automatikusan hozza létre a befizetési igényt az óra foglalásakor/kurzusra jelentkezéskor.

- tanulói ösvénynél csak akkor folytatható bármilyen haladás ha a diák jelezte befizetését, és a tanár visszaigazolta.
- hagyományos kurzus esetén be nem fizetett óráknál a tanulói fal új anyagjai lezárásra kerülnek.
- az előre tömbösített befizetésnél a tanuló jelezheti ezt, amit a tanár visszaigazol, de a tanár is jóváírhatja kérelem nélkül is.
- ezeket pedig egy összesített pénzügyi fülön meg is lehet tekinteni, a teljes bevételt, kik tartoznak még, és kik adtak be kérelmet a fizetés felülvizsgálatára.

---

9. **Tanári fájlkezelő**

Hogy a tanárnak ne kelljen többször feltöltenie ugyan azokat a fájlokat, létrehozni a feladatokat, dolgozatokat, vagy esetleg lecke anyagokat, lehetősége lesz egy fájlkezelőben feltölteni, létrehozni, rendszerezni ezeket. Aztán ezek beilleszthetőek

bárhová, új kurzus ösvényébe, hagyományos kurzus tanulói falára is, de akár másolatot is lehet készíteni egy adott helyre, ha például az adott diáknak személyre szeretnénk szabni a feladatot, anyagot.

Ha beadandó feladatról, feladatlapról van szó, azok láthatóak egy kurzus összesítő oldalon in, de egyénileg a diák tanulói falán egy külön fülön is, ahol látszanak az összesítők, és lehetőség adódik a javításra, visszajelzésre is.

---

10. Tanár-tanuló chat

A könnyű kommunikáció érdekében külön chat fülön tudnak a tanulók és a tanárok kapcsolatba lépni. Ha a tanulóknak megjegyzése lenne, vagy segítséget kérnének egy adott modullal/leckével kapcsolatban, az oda gépelt üzenet is itt kerül megjelenítésre és csatolja a linket az adott leckéhez, ahol az észrevétel volt.

---

2. Fejlesztési megfontolások

1. Tanári kapacitás korlátozása

Mivel egy tanárnak véges a konzultációkra alkalmas ideje, és akár ezt több kurzusban is szét kell osztani, ha több mindent oktat, vagy a tanár által, vagy a rendszer által csak bizonyos számú jelentkezőt kellene engednie, hogy biztosan mindenki megkaphassa a foglalt konzultációk számát.

---

2. Rendszer költségei

A platform fenntartásának finanszírozására szükséges lenne egy fenntartható használati díjazásra.

Ötletként felmerült százalékos bevétel, azaz a tanár által megadott árakra a platform ráteszi a megadott százalékot fenntartási költségként, DE ez a helyi fizetőkapu hiányában nehezen kivitelezhető.

Esetlegesen akkor előfizetési megoldásra lenne szükség?

---

3. Szülői fiók

Lehetne szülői fiókot is regisztrálni, ilyenkor a szülő létrehoz gyermeke számára egy korlátozott tanulói profilt.

A szülő:

- vehet fel új kurzusokat gyermekének
- kezeli a pénzügyi visszaigazolást
- látja gyermeke haladását
- közvetlen kapcsolatba tud lépni a tanárral

A gyermek:

- csak a felvett kurzusokat végezheti
- nincs rálátása a korlátozott funkciókra

Projekt megvalósítása

Kéttagú csapatként rengeteg nem várt megpróbáltatással találtuk szembe magunkat a projekt megvalósítása során.

Tudtuk, hogy a projekt mérete túlnyúlik a vizsgaremek elvárásain, de az ötletben láttunk potenciált későbbi piacra lépésre is, ezért előre igyekeztünk megtervezni az egészet, hogy a továbbfejlesztés minél könnyebben mehessen majd. Ezért a teljes elképzelés összeírását követően körvonalaztuk melyek azok a funkciók, amelyeket mindenképpen szeretnénk implementálni a vizsgaremek beadási határidőjéig elkészülő prototípusba. Olyan funkciók, modulok is belekerültek mind az ötletelési dokumentumba, az adatbázis tervbe és néhány végpont is, amelyek nem kerültek felhasználásra a beadott projektmunkában, vagy egy egyszerűsített verzió került beépítésre (pl. chat funkció nem valós idejű, de demonstrációra alkalmas).

A prototípusban a projektötlet alapján a vizsga beadásáig megvalósítandó részek: Tanár-Tanuló regisztráció és bejelentkezés, magántanári funkciók (keresés, kurzus felvétel, tanulói fal) teljes implementálása a fájlfeltöltést igénylő elemek kivételével (kivételek: beadandók beadása, fájlfeltöltés a tanulói falra, fájlkezelő).
Megvalósításra nem került sor, vagy egyszerűsített módon lett:

- fizetőkapu → egyéni bevalláson és visszaigazoláson alapuló rendszer
- fájlkezelő, fájlfeltöltés → nem került megvalósításra a frontenden, de az adatbázis készen áll rá
- chat, értesítések → nem valós idejű
- AI funkciók → nem kerültek megvalósításra
- közösségi funkciók → nem kerültek megvalósításra
- feladatlap kreátor → nem kerültek megvalósításra, de az adatbázis készen áll rá
- tanulói ösvények, videóleckék, haladás → nem kerültek megvalósításra, de az adatbázis többnyire készen áll

---

A csapatmunka megvalósítására a projekt prototípusának elemeit felbontottuk egybefüggő modulokra, és úgy határoztunk, hogy felváltva dolgozunk majd a backenden és a frontenden. Ez nem így valósult meg, mivel hamar rájöttünk, hogy sajnos párhuzamosan nem tud haladni

a fejlesztés, mert a backendhez szükséges az adatbázis, de az pedig a robosztussága miatt nem bővíthető könnyen.

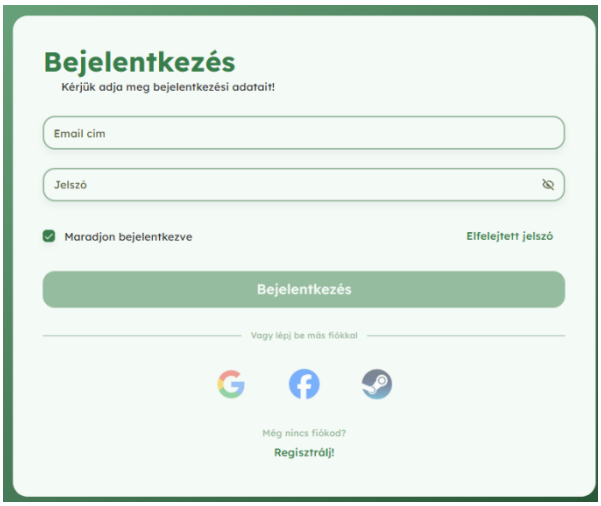
Ezért az első – bejelentkezés és regisztrációs – fázist követően rögzítettük a részeket, Béláé lett a frontend fejlesztése, míg Péteré a backend, adatbázis tervezése, és a frontendhez wireframe-ek készítése.

A megkésett kezdést követően eleinte próbálkoztunk projekt haladási tervvel, de ez nem volt tartható a közelgő beadási határidő miatt, így a frontend és backend fejlesztése különvált míg nem sikerült a backendet az adatbázissal teljesen összekötni.

Végül az eltervezettek többségét sikeresen megvalósítottuk, a keletkező rengeteg akadály ellenére.

Felhasználói Dokumentáció

Bejelentkezés/regisztráció:



A bejelentkezés ablakon, a már létrehozott fiókokkal, lehet a honlapba bejelentkezni. Ha még nincs a felhasználónak fiókja a regisztráció oldalon tud létrehozni egy újat.

Ha véletlen a felhasználó elfelejtette a jelszavát, az elfelejtett jelszó oldalon hozhat létre egy újat.

Regisztrálni csak új és még nem létező felhasználót lehet. Amennyiben megadta az új email címet jelszóval beállítva, a felhasználó át lesz irányítva egy új oldalra ahol megadhatja a személyes adatait.

Tanulói Oldal:

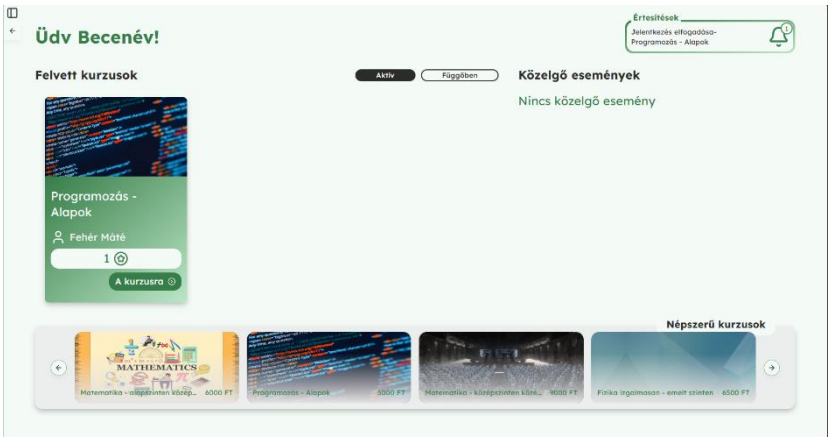
Amennyiben a felhasználó egy tanulói fiókkal jelentkezett be, a külön számára elkészített oldalra vezeti át.

Itt minden tevékenységet elvégezhet ami a tanuló hatáskörébe tartozik.

Tanulói oldalsáv:
Ez az oldalsáv azt a célt szolgálja hogy a felhasználó szabadon válogathasson a számára felkínált oldalak közül.

Pontok:

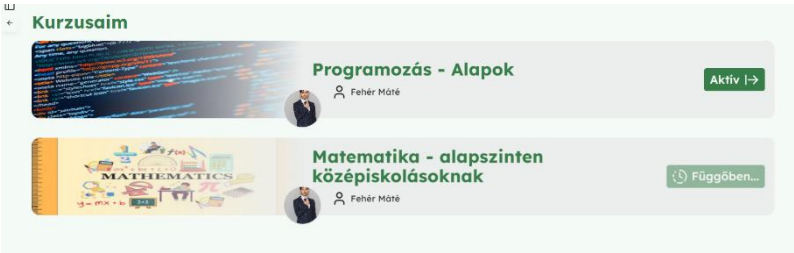
- Főoldal: Általános információk
- Kurzusaim: Összes kurzus amire a tanuló jelentkezett.
- Üzenetek: Személyes chat ablak, az összekötött felhasználók között.
- Órarend: Vízuális mutatása az elkövetkező óráknak
- Kereső: Új kurzus keresése, és az arra való jelentkezés.
- Értesítések: Új és régi értesítések a fiókkal kapcsolatban.
- Pénzügyek: Összes pénzügy felsorolása amit a tanuló kezdeményezett.
- Beállítások: Személyes adatok módosítása itt lehetséges.
- Kijelentkezés: Kijelentkezik a jelenlegi felhasználóból.



Tanulói főoldal:

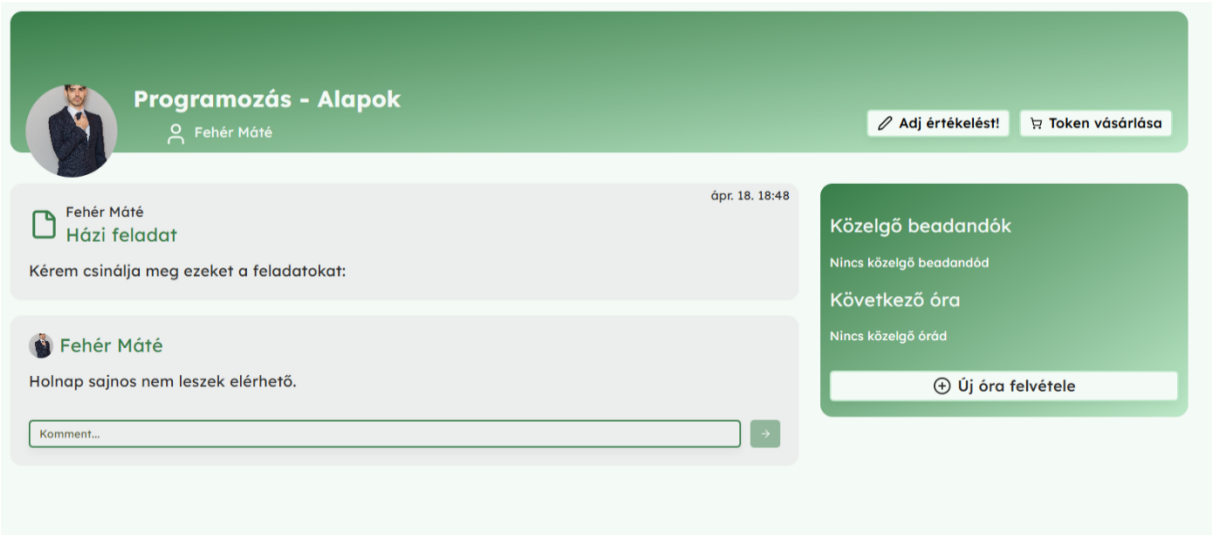
1. Értesítések
2. Felvett/Függőben lévő kurzusok
3. Közelgő események
4. Népszerű kurzusok

Általános információk amik fontosak lehetnek egy tanulónak.



Tanuló kurzusai:

Tanuló már aktív és felvett oldalait találhatja itt.

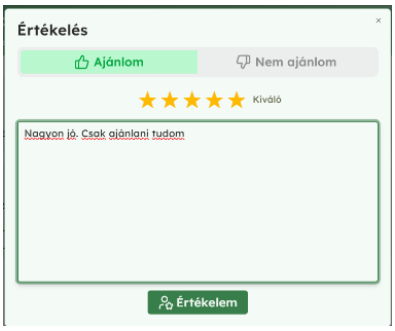


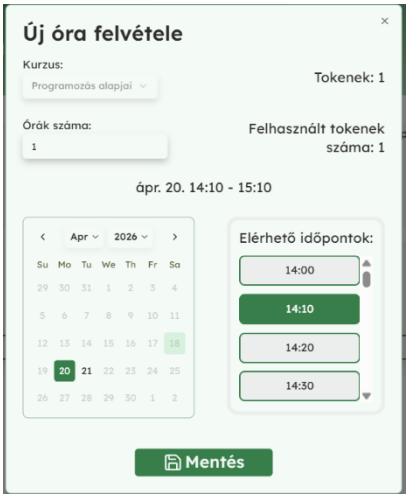
Kurzus fal:

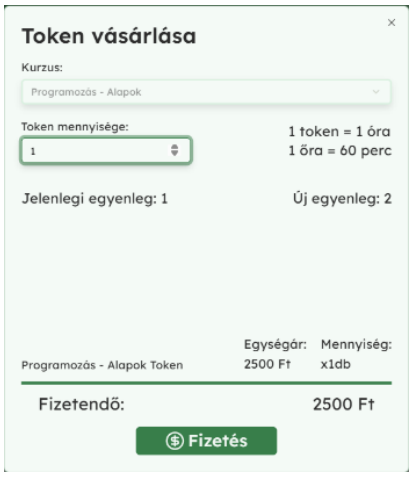
Ezen az oldal található a kurzushoz tartozó összes információ/tudnivaló.

Itt lehet:

- Értékelni a kurzust:
- Tokeneket vásárolni: Az új tokent csak akkor adja hozzá amint a tanár elfogadta, hogy sikeres fizetés történt.
- Új órát felvenni: Csak olyan idő pont választható amit a tanár élőre jelezte hogy szabad. És elegendő tokennel.

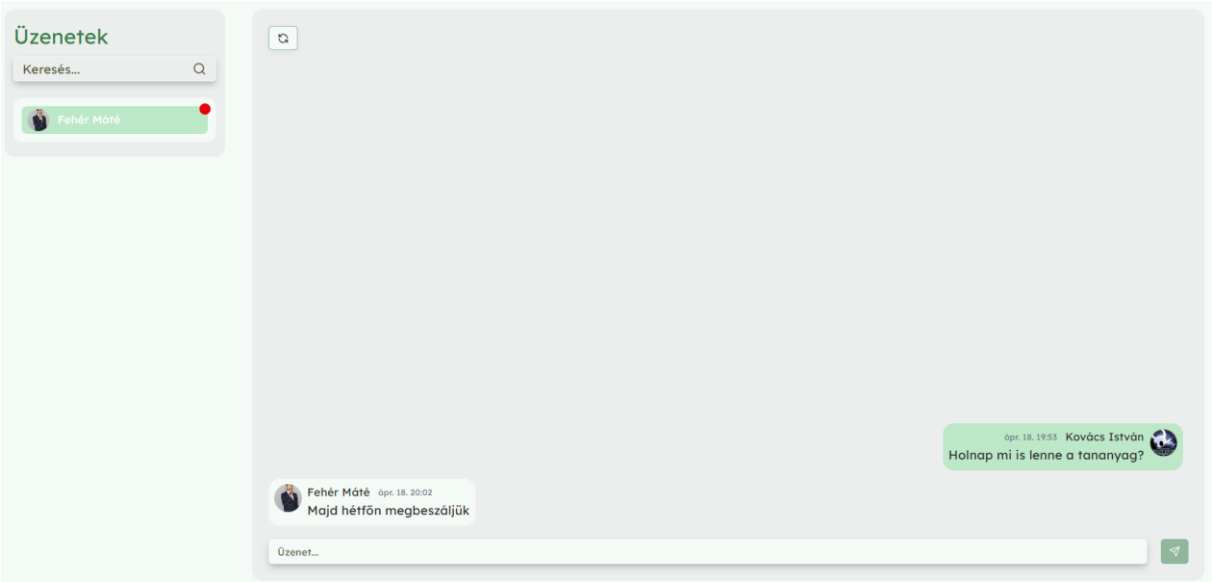






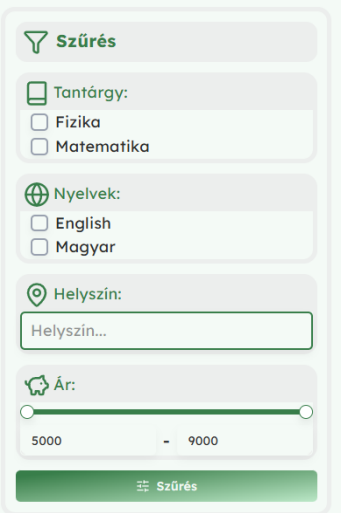
Ezeken felül az oldalon még interaktálhat a posztokkal és beadandókkal amit a tanárra rakott ki. Itt lehet kommentelni az egyes posztokra.

Üzenetek:



Ezen az oldalon egy magán chaten kommunikálhat a tanuló a tanárjaival. Az eddig olvasatlan üzeneteket a felhasználó fiókjánál jelzi.

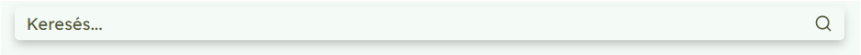
Kereső:

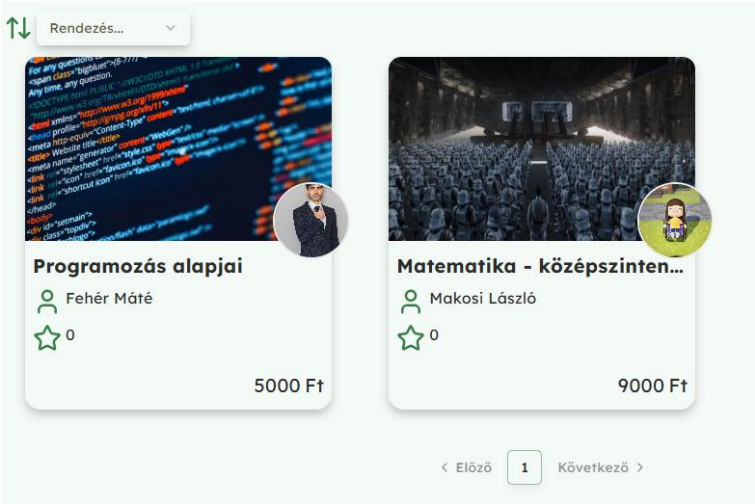


Az oldalsó sávon a felhasználó beállíthat a számára megfelelő szűrési feltételeket.

A szűrés gombra kattintva elő alkalmazza is a megjelentetett kurzusokra.

Ezen felül még név alapján is kereshet a felhasználó.



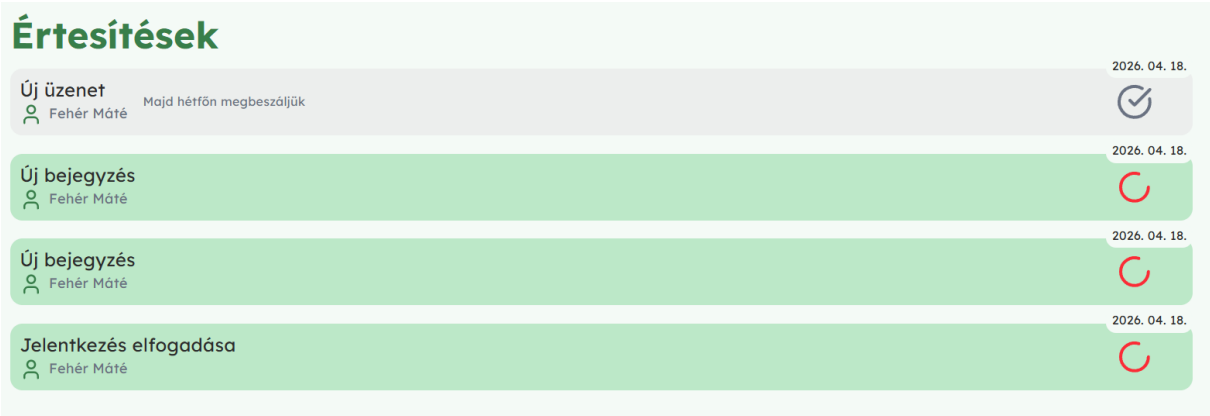


A kurzusokat kártyákként jeleníti meg, ezek vezetnek át a kurzus oldalára.

A kártyákon általános információk találhatóak meg a kurzusról

A kurzus oldalán jelentkezhet a felhasználó, ahol egyben az összes fontos információt is találja a kurzussal kapcsolatban.

Értesítések:

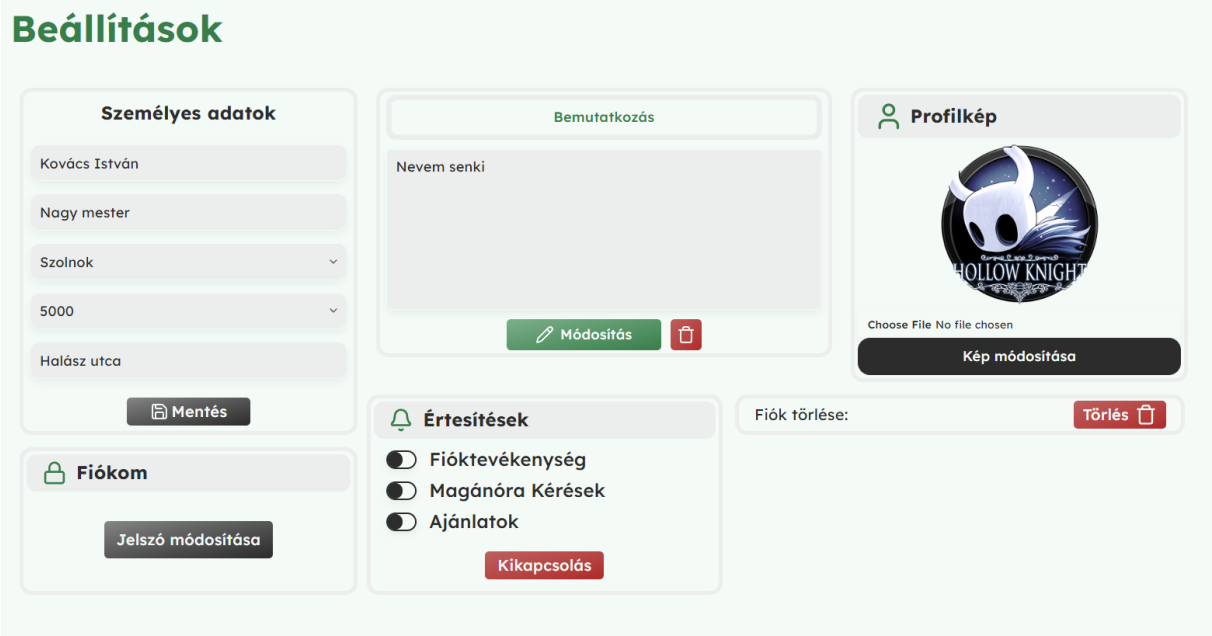


Ezen az oldalon látható az összes értesítés ami a felhasználóhoz tartozik. Külön jelezve az elolvasott és a még nem olvasott értesítéseket.

Pénzügyek:

A felhasználó összes pénzügye megtalálható ezen a honlapon. Itt láthatja a függőben lévő/Elfogadott/Elutasított tranzakcióit.

Beállítások:



Ezen az oldalon módosíthatja a felhasználó az összes személyes adatát.

Ezek:

- Személyes adatok
- Bemutatkozás
- Profilkép
- Jelszó
- Fiók törlése

Tanári Oldal:

Amennyiben a felhasználó egy tanári fiókkal jelentkezett be, a külön számára elkészített oldalra vezeti át.

Itt minden tevékenységet elvégezhet ami a tanár hatáskörébe tartozik.

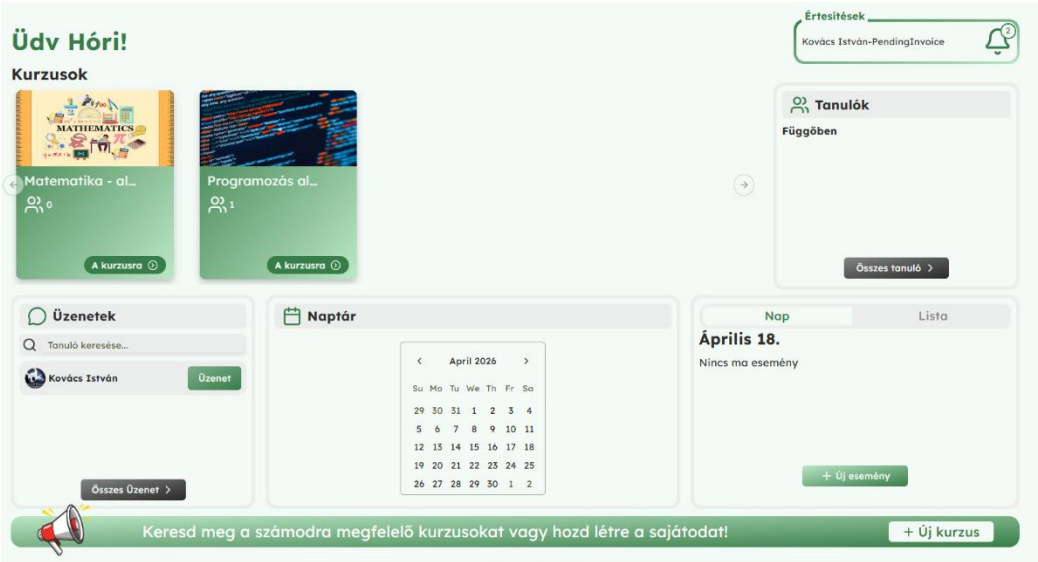
Tanári oldalsáv:

Ez az oldalsáv azt a célt szolgálja hogy a felhasználó szabadon válogathasson a számára felkínált oldalak közül.

Pontok:

- Főoldal: Általános információk
- Kurzusaim: Összes kurzus amire a tanuló jelentkezett.
- Tanulóim: A tanárhoz tartozó összes tanuló
- Üzenetek: Személyes chat ablak, az összekötött felhasználók között.
- Órarend: Vízuális mutatása az elkövetkező óráknak
- Értesítések: Új és régi értesítések a fiókkal kapcsolatban.
- Pénzügyek: Összes pénzügy felsorolása amit a tanuló kezdeményezett.
- Beállítások: Személyes adatok módosítása itt lehetséges.
- Kijelentkezés: Kijelentkezik a jelenlegi felhasználóból.

Tanári főoldal:

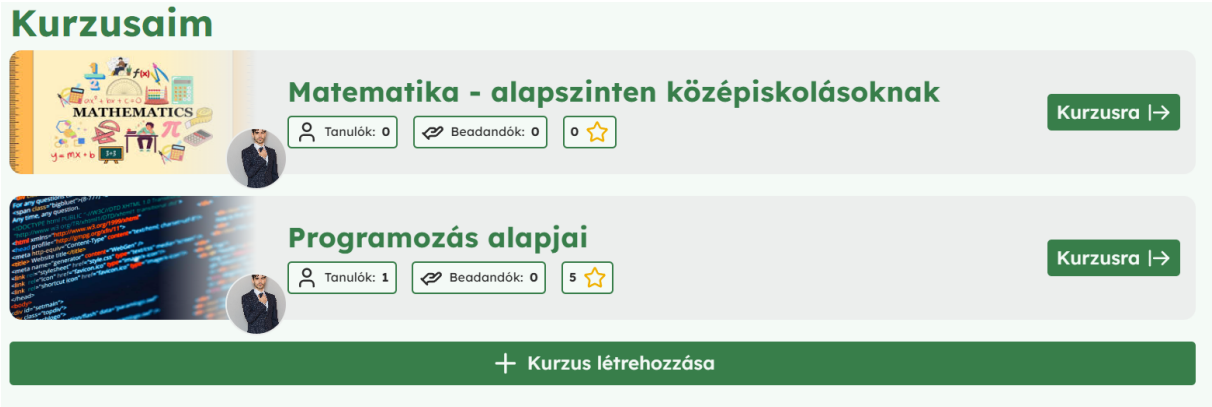


A főoldalon a tanárhoz tartozó legfontosabb információkat találhatjuk. Gyors elérést biztosítunk a leggyakoribb tevékenységekhez.

Ezek:

- Kurzusok
- Függőben lévő tanulók
- Üzenetek
- Naptár/közelgő események

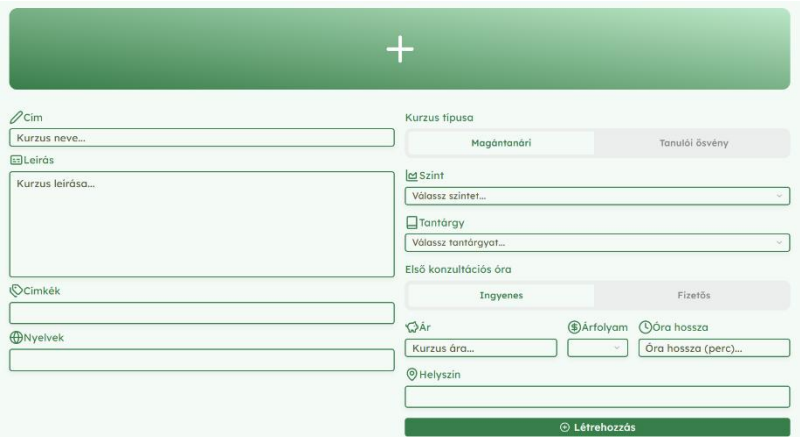
Kurzusaim:



A felhasználó választhat a már létrehozott kurzusai között, és újat is hozhat létre.

Új létrehozása:

A kurzushoz tartozó összes információ megadható, amit a program ellenőriz is, így nem adható le hibás adattal.

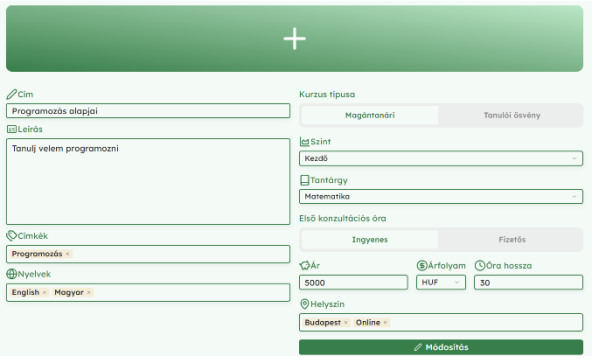


A kurzusra áttérve több oldal közül is választhat a felhasználó:

- Kurzus profil módosítása
- Kurzushoz tartozó tanulók
- Kurzus áttekintése (Ahogy a tanulóknak lesz megjelenítve)

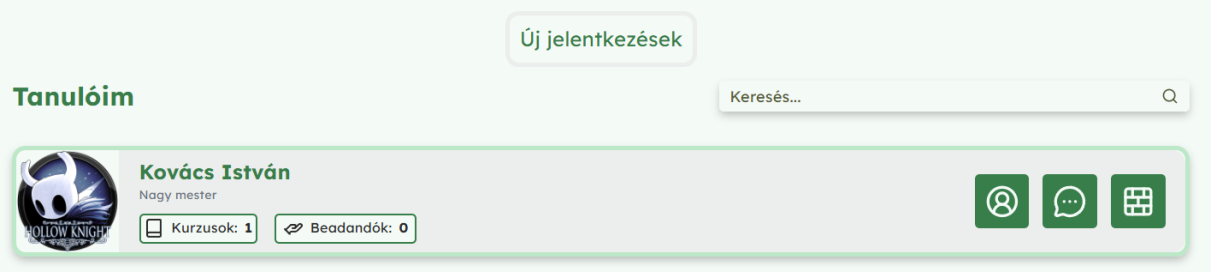
Módosítás:

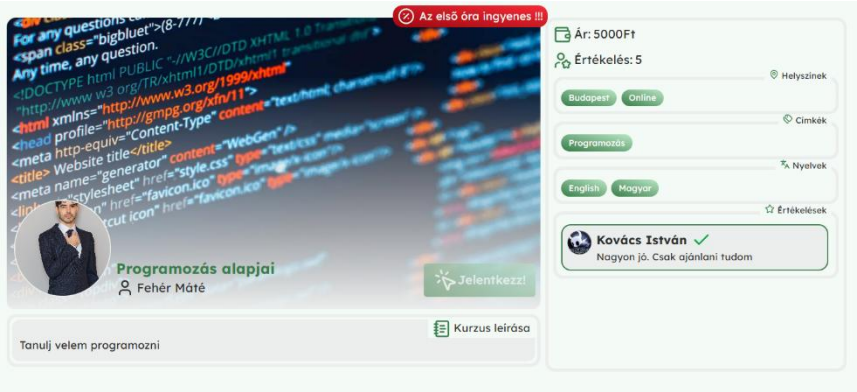
Automatikusan kitölti a kurzus adatait, amit a felhasználó szabadon változtathat.



Tanulók:

Összes tanuló a kurzusban:

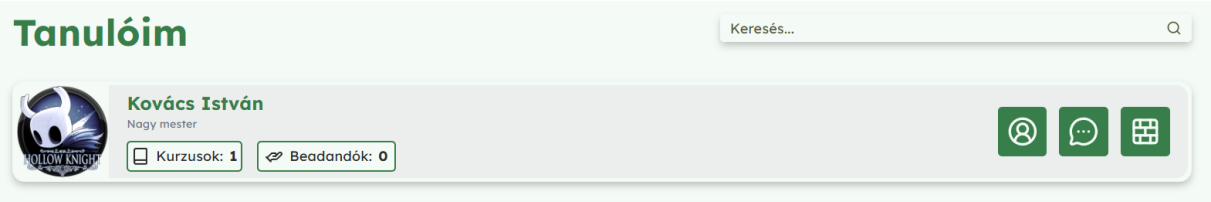




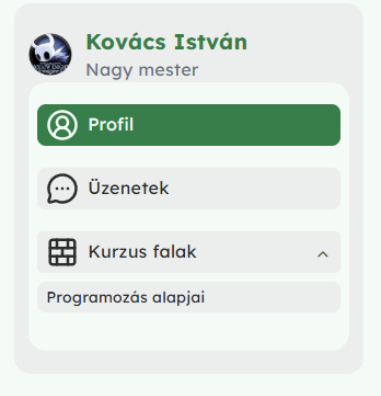
Áttekintés:

A felhasználó megtekintheti mit fognak felhasználók látni amikor megnyitják a kurzust.

Tanulóim:

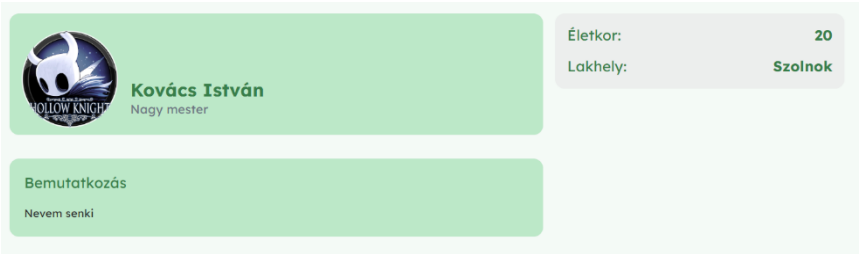


A tanár összes tanulója megtalálható, és név alapján szűrhető.



A három opció közül az egyikre kattintva átmehet a tanulói oldalra.

- Profil
- Üzenetek
- Kurzus falak: (az összes kurzus külön amire a tanuló jelentkezett)



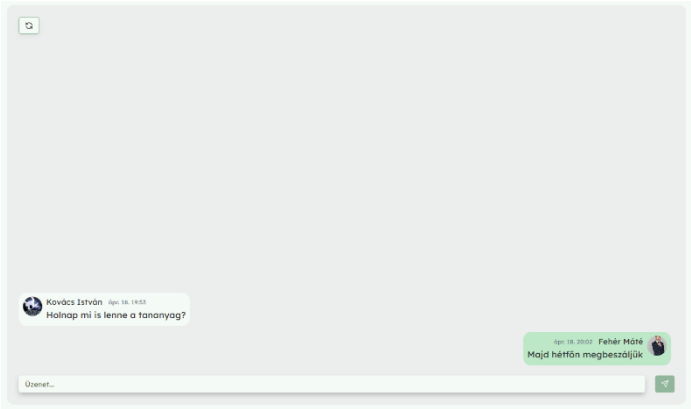
Profil:

A tanuló általános adatai.

Üzenetek:

Személyes chat a tanulóval.

(Azonos az üzenetek oldal alatt található chat ablakkal)

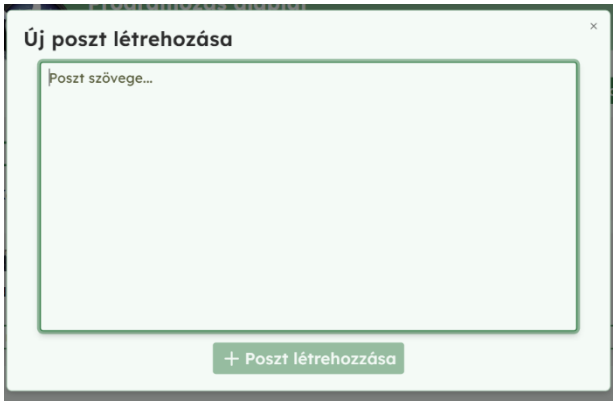


Kurzus fal:

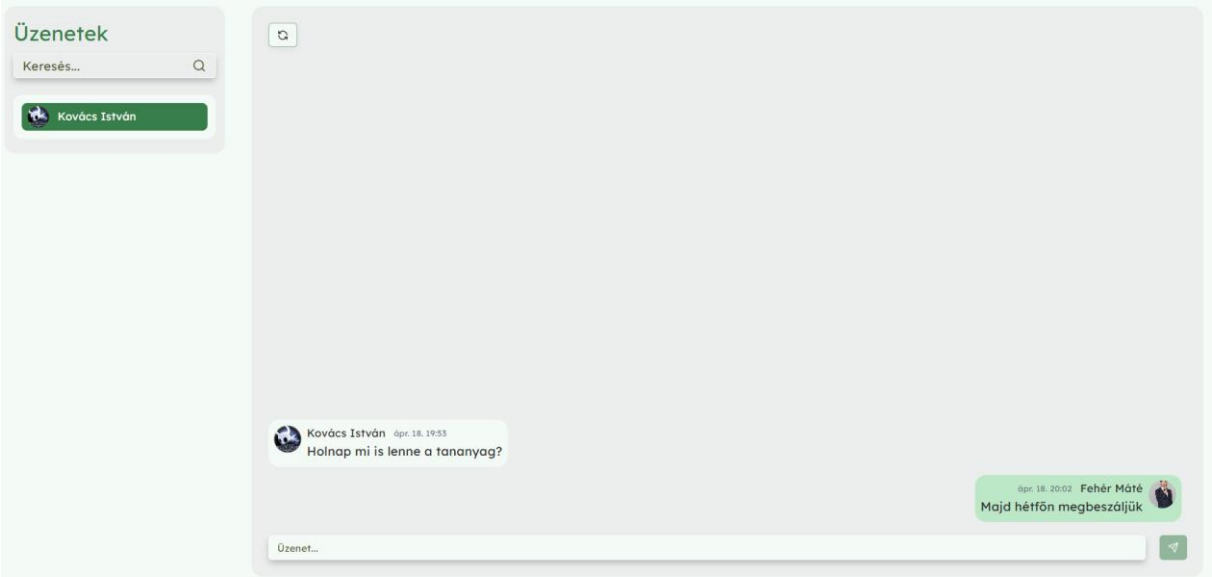
A tanár ezen az oldalon kezelheti a tanulóval kapcsolatos kurzus falát.

Létre hozhat új posztokat/beadandókat

És ezekre kommentálhat is.



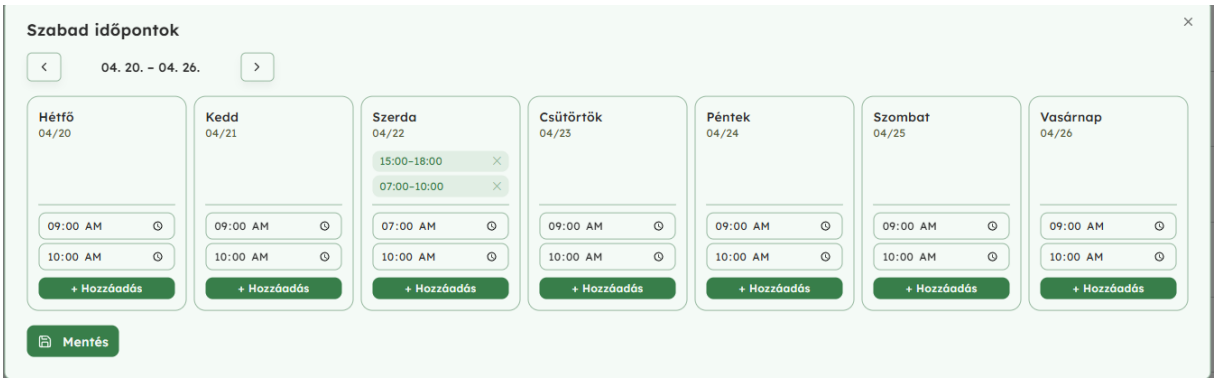
Üzenetek:



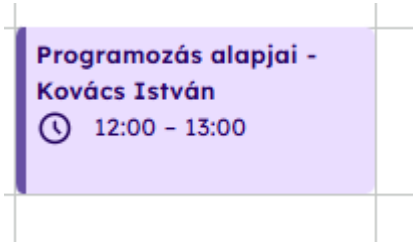
Tanulóhoz hasonlóan a személyes magán szobákat találhatja meg a felhasználó ezen az oldalon.

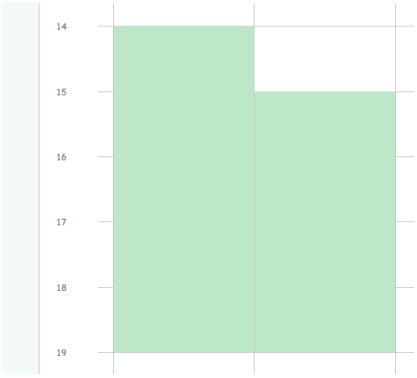
Órarendem:

A felhasználó ezen az oldalon megtalálhatja az összes órát amit a tanuló foglalt és itt adhat hozzá elérhető időpontokat.

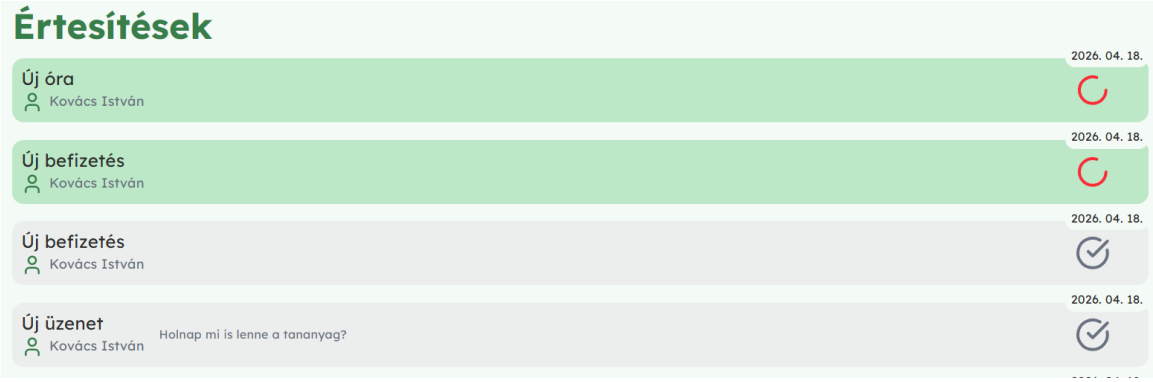


Szabad időpontok beállítása alatt a tanár létrehozhat zónákat amiben a tanulói foglalhatnak. Ezen zónákon kívül viszont nem lehet.





Értesítések:

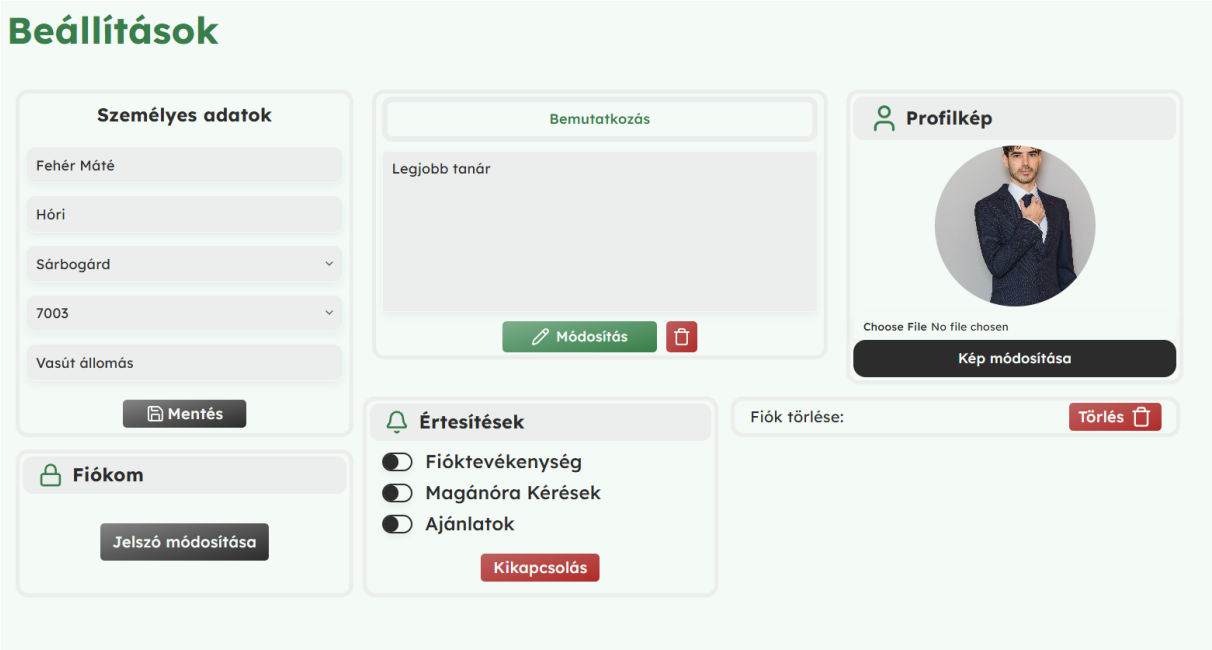


Ezen az oldalon látható az összes értesítés ami a felhasználóhoz tartozik. Külön jelezve az elolvasott és a még nem olvasott értesítéseket.

Pénzügyek:

Ezen az oldalon láthatja a tanár a beérkező pénzügyeket, amiket elfogadhat vagy elutasíthat annak függvényében, hogy a tanuló valóban kifizette-e az összeget.

Beállítások:



Frontend Felhasználási útmutató

Célunk: A frontend tervezése alatt, mindvégig szem előtt tartottuk, hogy egy letisztult és könnyen felhasználható design-t hozzunk létre. Mindvégig figyeltünk rá, hogy a modern design elveket kövessük.

Felhasználói folyamat:

1. **Bejelentkezés/Regisztráció:** Amíg a felhasználónak nincs profilja regisztrálhat egy újat. 2 szerepkör közül választhat: Tanuló/Tanár. Szerepkörtől függően más-más oldalakat jelenít meg.
2. **Főoldal:** Egy általános információt ad vissza a felhasználónak, szerepkörétől függően.
3. **Kurzusok:** Megjeleníti a felhasználó aktív kurzusait.
   - **Tanuló:** Kurzusról a kurzus falra lehet átmenni. Ezen az oldalon adhat ki a tanár posztot és beadandód. Itt lehet ezekkel interaktálni/kommentelni. Ezen felül itt lehet értékelni a kurzust, vásárolni tokeneket a kurzushoz, új órát foglalni a tanár szabadidőpontjától függően.
   - **Tanár:** Itt három külön oldalt találhat a felhasználó.
     - **Kurzus módosítása**
     - **Tanulók áttekintése**
     - **A kurzus áttekintése** (Amit a tanulók fognak látni)
4. **Tanulóim (Tanár):** A tanár összes tanulója megtalálható itt, kereshetően. 3 oldal nyitható meg minden tanulónál:
   - **Profil:** A tanuló általános információi
   - **Üzenetek:** Személyes üzenetek a tanulóval
   - **Kurzus fal:** A tanuló kurzusainak falát itt lehet elérni. Posztokat és beadandókat létre lehet hozni.
5. **Üzenetek:** Ezen az oldalon érhető el a felhasználó személyes üzenetei egy adott felhasználóval (Tanár – Tanuló kapcsolat)
6. **Órarend:** A naptárban megjeleníti a hónapban lévő következő órákat.
   - **Tanár plusz funkciói:** Beállítható a tanár elérhető időszakai, ahová a diák foglalhat. Ezeket a naptár megjeleníti sávokban.
7. **Keresés (Tanuló):** Ezen az oldalon található az összekurzus amire a tanuló jelentkezhet. Ezeket a kurzusokat szűrheti (Név/Tantárgy/Nyelv/Helyszín/Ár alapján), és rendezheti is. Kurzusra kattintva átvisz a kurzus oldalára ahol a kurzus általános információit, a tanár profilját(Tanárra kattintva) láthatja. Itt jelentkezhet a kurzusra. (Amit a tanár elfogadhat és elutasíthat).

8. **Értesítések:** Ezen az oldalon található a felhasználóhoz tartozó összes értesítés.
9. **Pénzügyek:** Itt tekinthető meg a felhasználó összes pénzügyi tevékenysége megtekinthető.

Software Architecture Document (English)

---

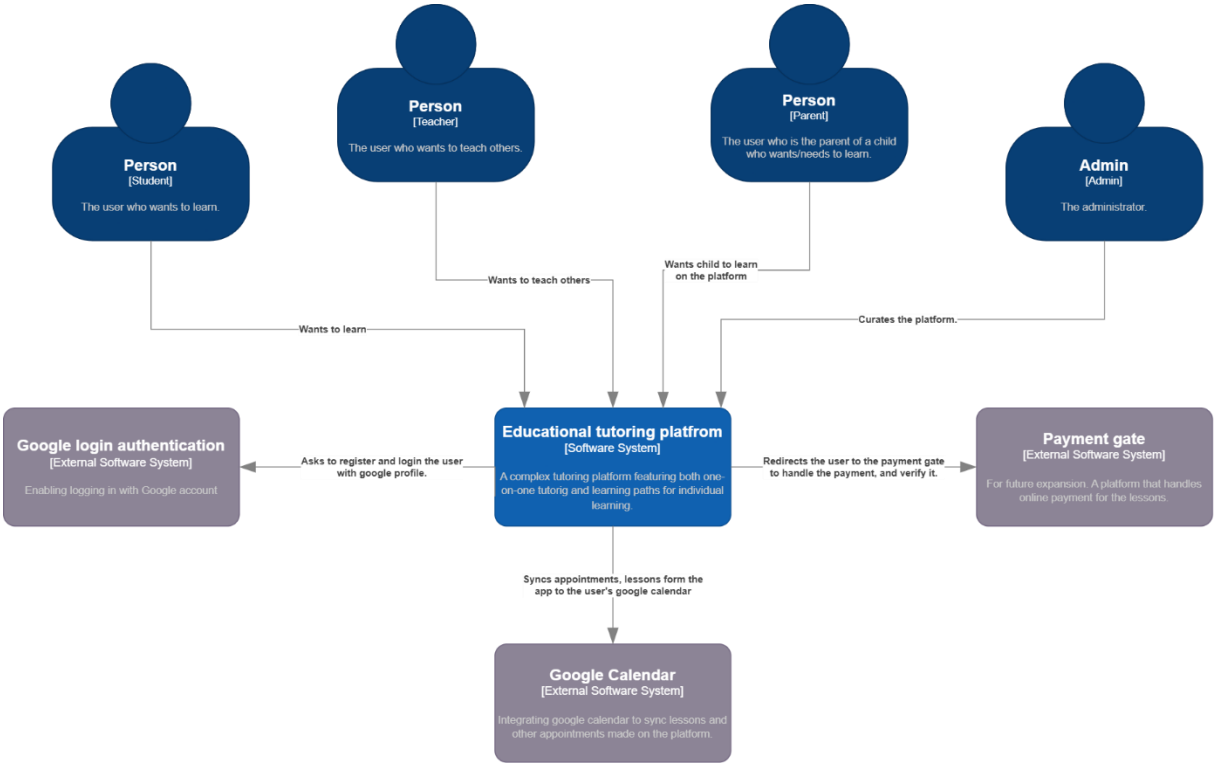
1. Tech-Stack

- Frontend: NEXT.js – routing, Proxy/Backend-For-Frontend
  - TailwindCSS
  - ShadCN (UI library)
- Backend: ASP.NET (Identity, EF Core) – Modular Monolith
- SQL: PostgreSQL – structured data
- Container(s): Docker + Linux
- File storage: Linux Docker – binary files (PDF's, images, videos)

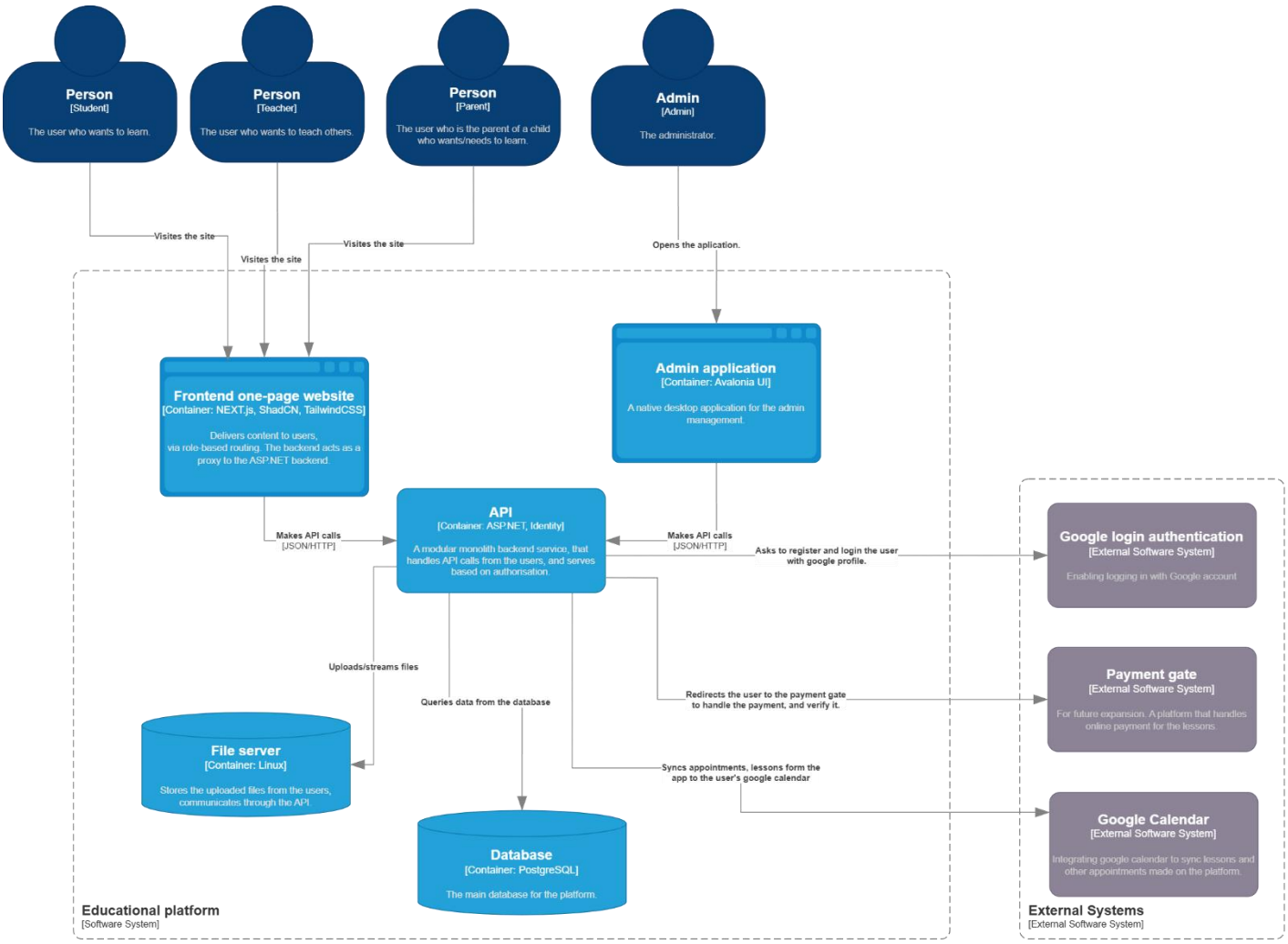
---

2. C4 Diagram

- Level 1:

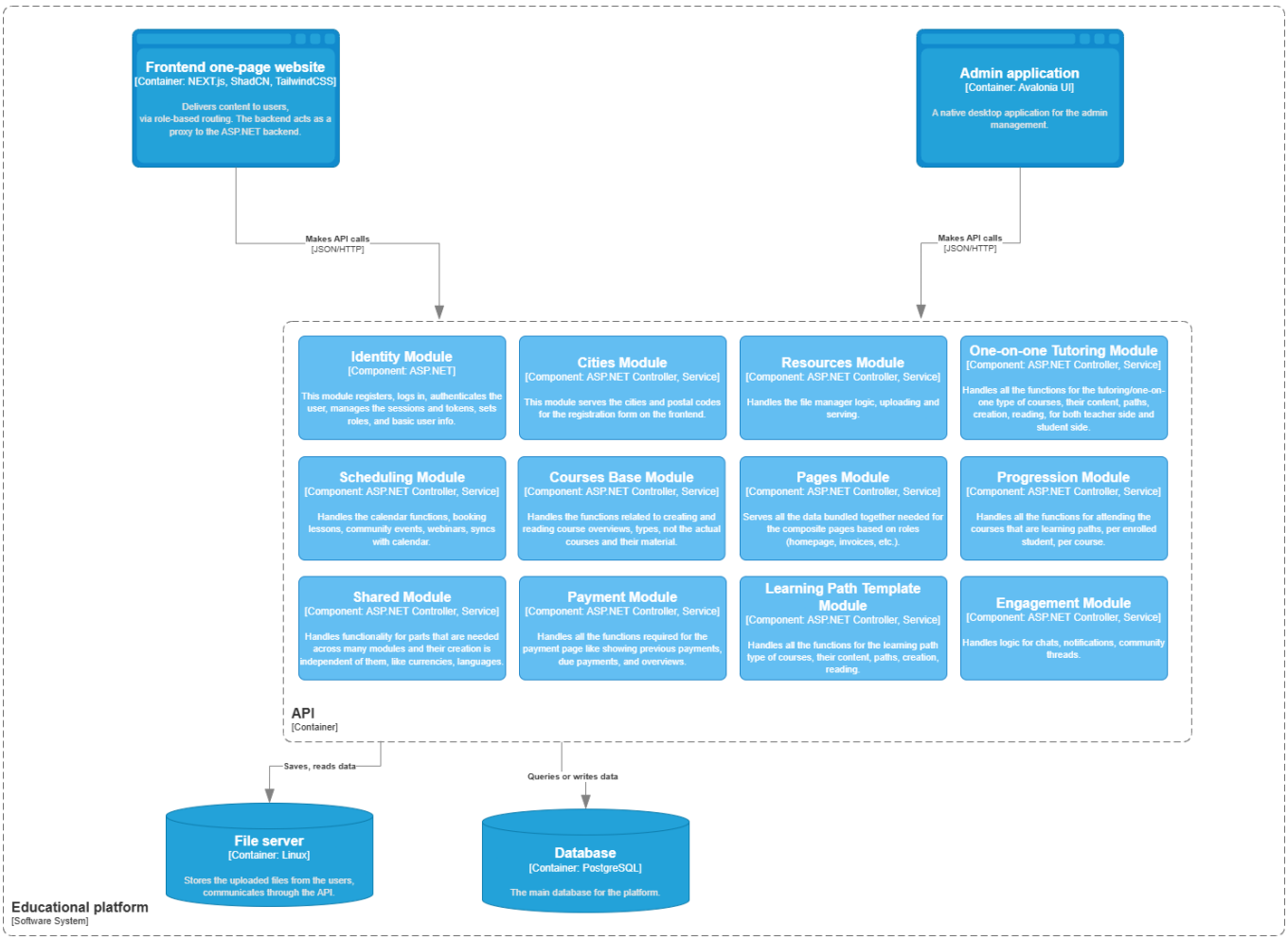


- Level 2:



  - **Frontend:** Delivers content to users, via role-based routing. The Next.JS backend acts as a proxy to the ASP.NET backend using a Backend-for-Frontend pattern, reducing attack surface by not exposing the user directly to the API.
  - **API:** A modular-monolith backend service, for later moving onto a microservice-architecture, that handles API calls from the users, and serves the frontend based on authorization.
  - **File Server:** Stores the uploaded files from the users, communicates through the API.
  - **Database:** The main relational database for the platform.
  - **External systems** (not yet implemented in the prototype)**:**
    - **Google Login Auth:** Enabling logging in with a Google Account for convenience.
    - **Google Calendar:** Integrates with the built in scheduling API module, lessons and other appointments show up in the synced calendar too.
    - **Payment gate:** For future expansion. A platform that handles online payment for the lessons.

- Level 3:



- **Modules implemented in the prototype:**
  - **Identity Module:** This module registers, logs in, authenticates the user, manages the sessions and tokens, sets roles, and basic user information.
  - **Cities Module:** This module serves the cities and postal codes for the registration form on the frontend.
  - **Resources Module:** Handles the file manager logic, uploading and serving.
  - **Scheduling Module:** Handles the calendar functions, booking lessons, community events, webinars, syncs with calendar.
  - **Courses Base Module:** Handles the functions related to creating and reading course overviews, types, not the actual courses and their material.
  - **Pages Module:** Serves all the data bundled together needed for the composite pages, based on roles (homepage, invoices, etc.).
  - **Tutoring Module:** Handles all the functions for the tutoring/one-on-one type of courses, their content, creation, reading, for both teacher side and student side.

- **Shared Module:** Handles functionality for parts that are needed across many modules and their creation is independent of them, like currencies, languages.
- **Engagement Module:** Handles logic for chats, notifications, community threads.
- **Payment Module:** Handles all the functions required for the payment page like showing previous payments, due payments, and overviews. For the prototype a self-declaration system was implemented, the student has to order a number of tokens, then pay in the price in some off-platform way, then the teacher can either mark the transaction successful, or failed. In the future a payment gate is planned to be implemented, making the system more centralized.

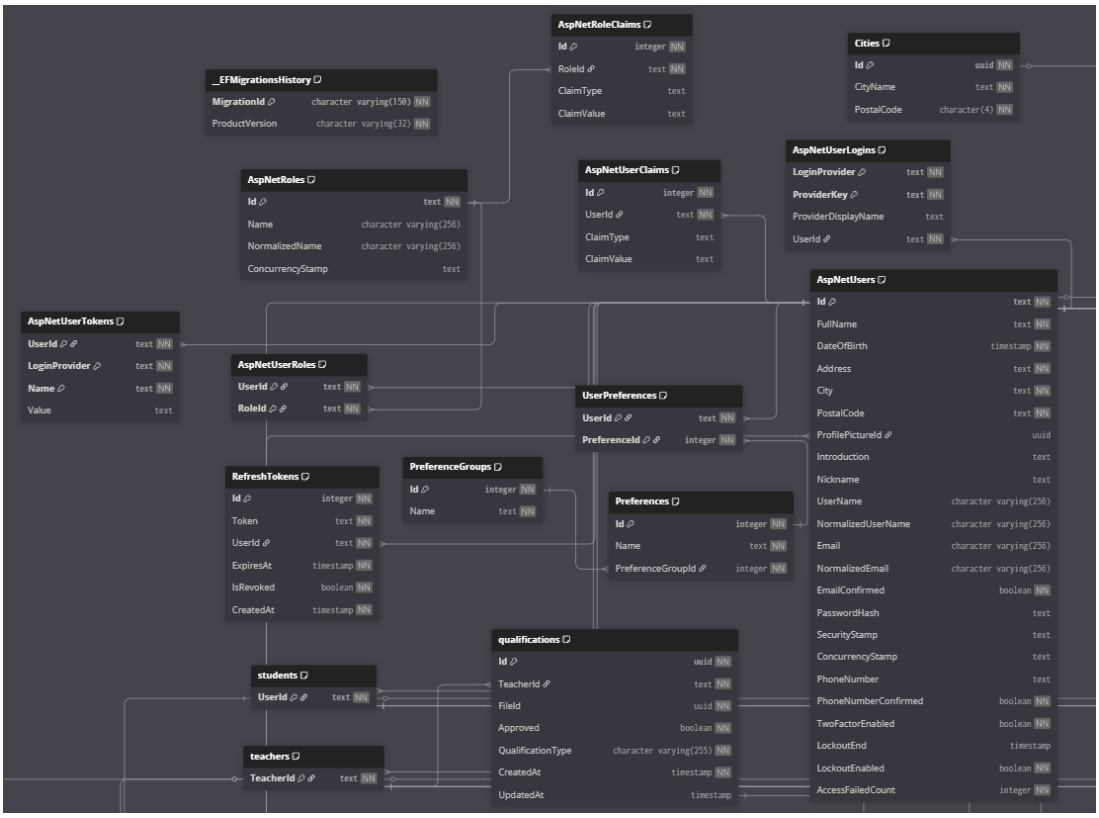
- **Modules planned for future development:**
  - **Learning Path Template Module:** Handles all the functions for the learning path type of courses, their content, paths, creation, reading.
  - **Progression Module:** Handles all the functions for attending the courses that are learning paths, per enrolled student, per course.

---

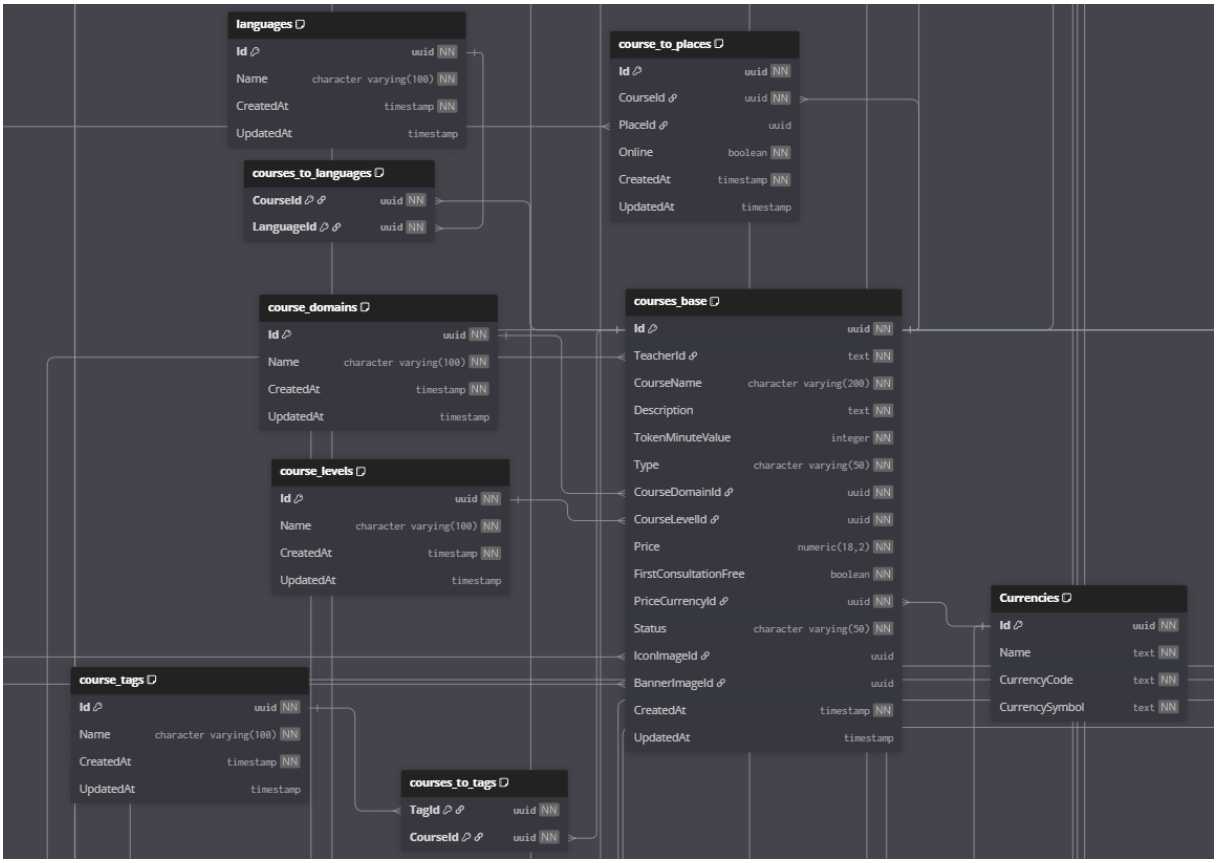
3. Database Schema

As data is the foundation of this system, we started by creating the robust database schema, implementing all planned functions including the ones that are not part of the first prototype planned to be finished for this exam, so that later we can safely build those on top, otherwise missing fields and keys would cause the entire database to be reinitialized. There are 53 tables in total, and they are organized into modules matching those in the backend architecture.

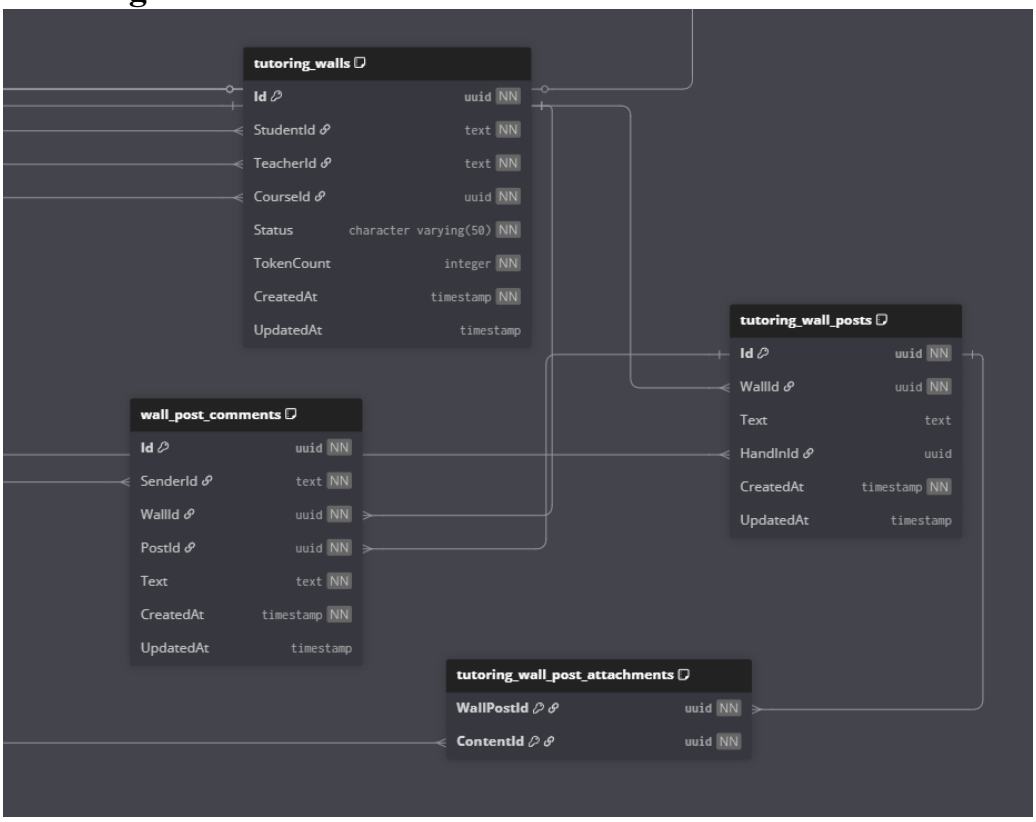
Identity and Cities Module



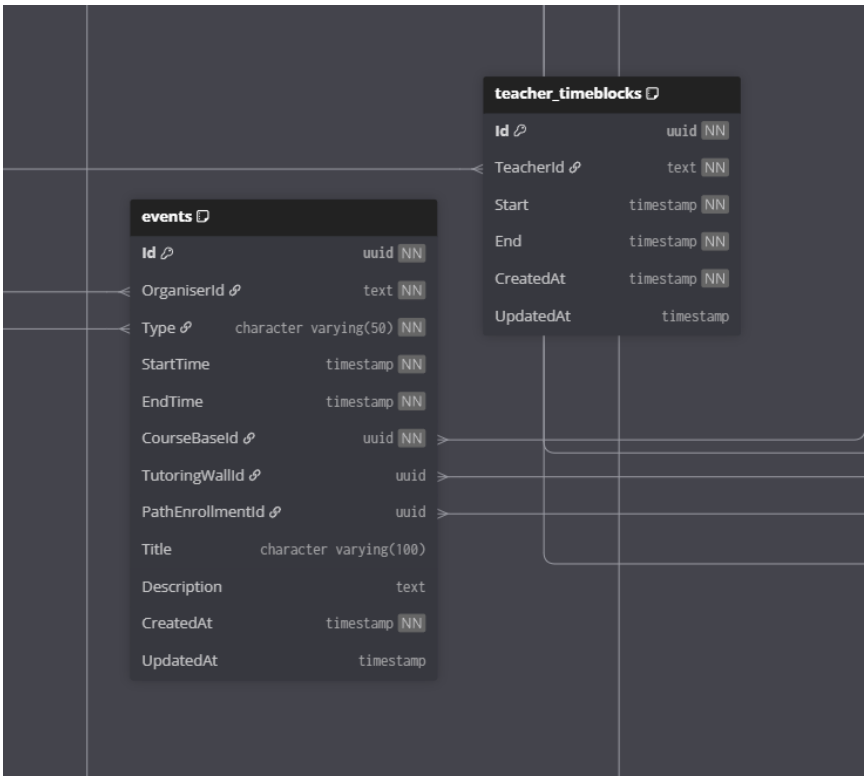
CourseBase module



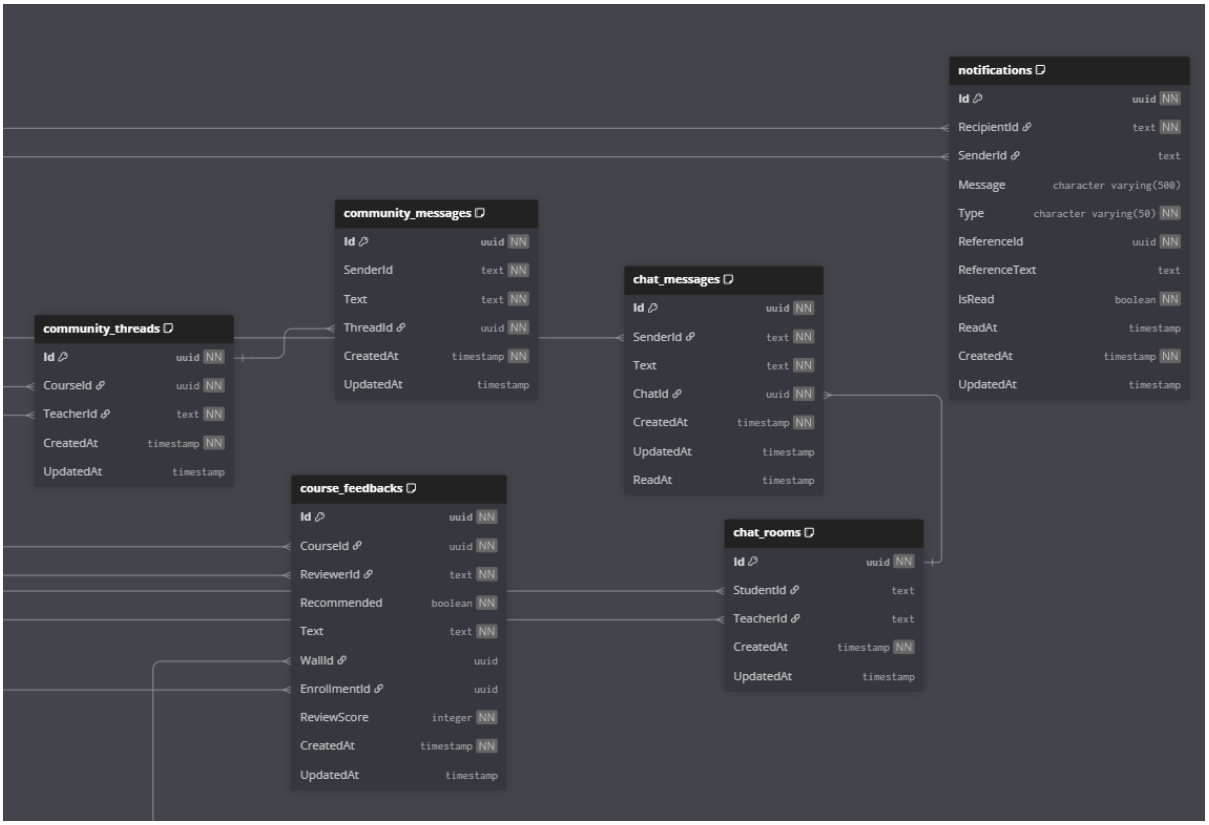
Tutoring module



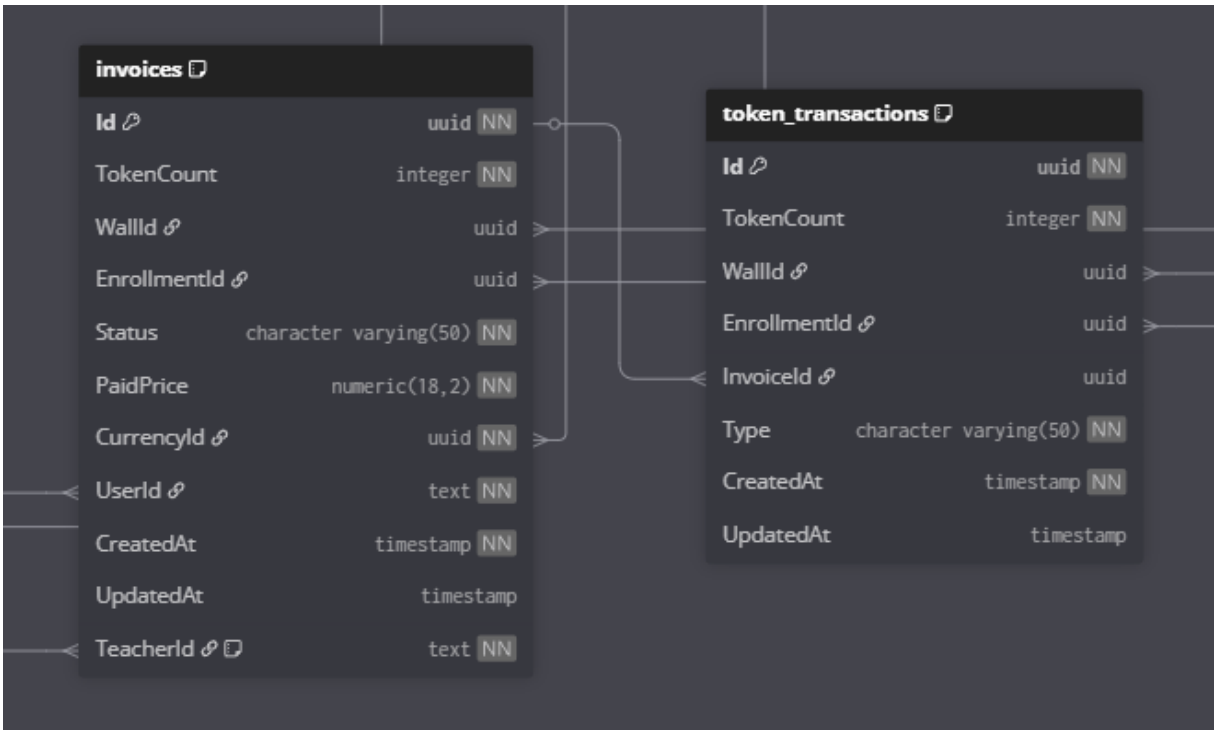
Engagement module



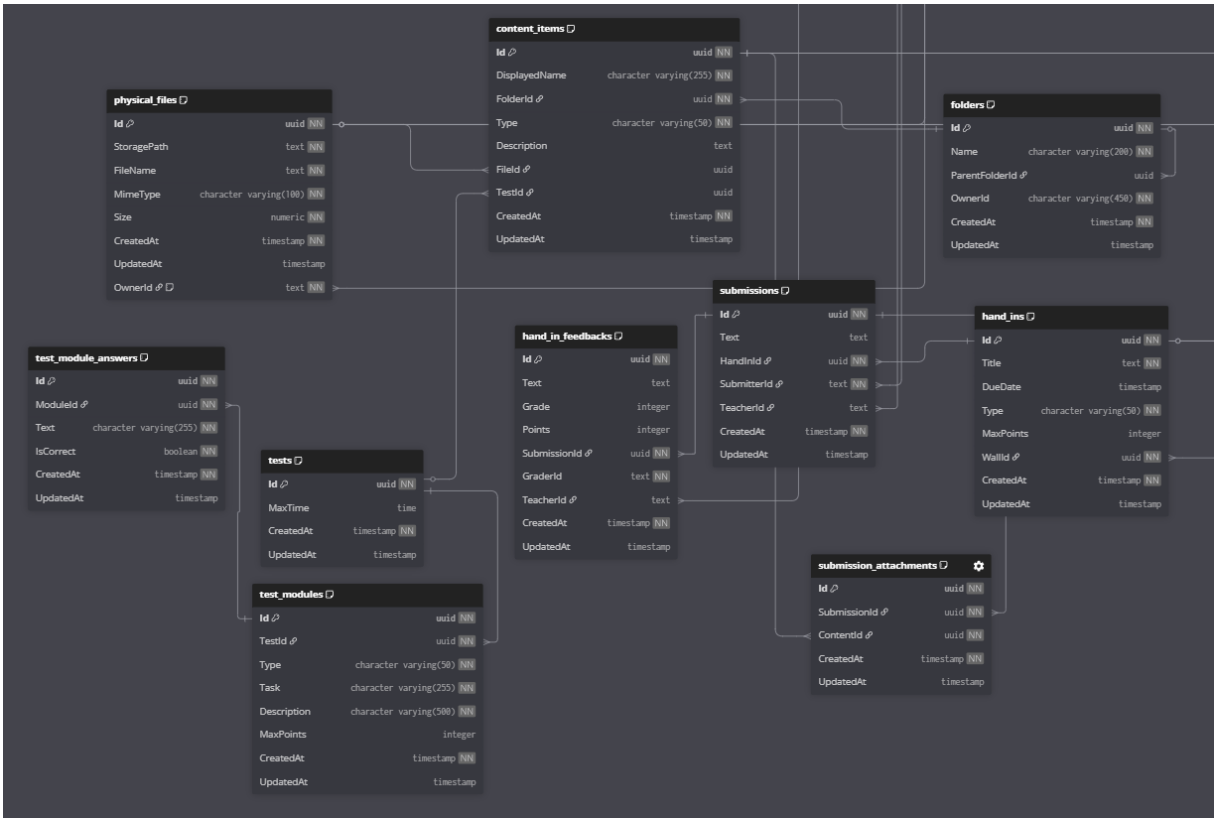
Scheduling Module



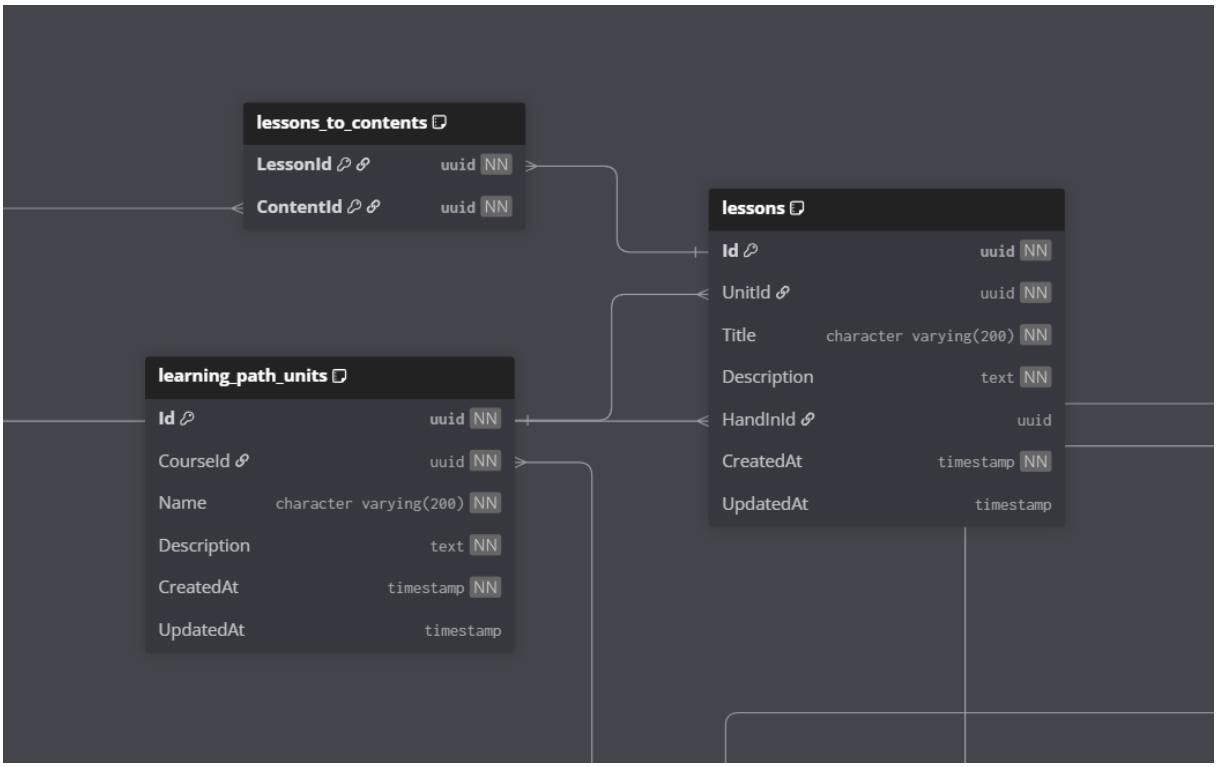
Payment Module



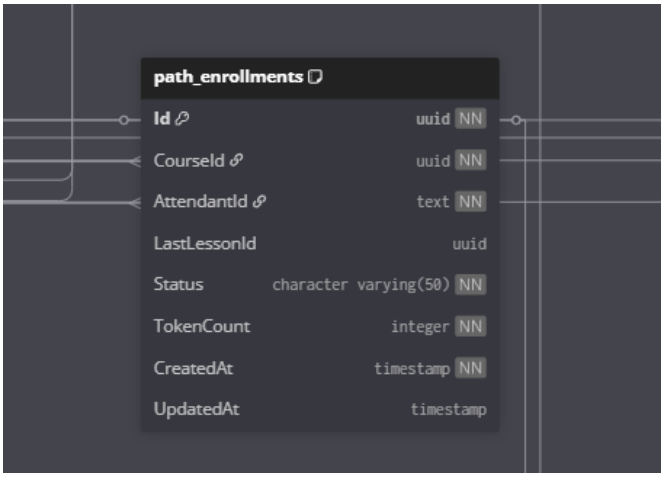
Resources Module



LearningPath Template Module



Learning Path Progression Module



4. API endpoint documentation

Method	Endpoint	Description
Auth — Authentication & Session Management		
POST	/api/auth/register	Register a new user account with email and password
POST	/api/auth/login	Authenticate with email/password, returns access and refresh tokens
GET	/api/auth/logout	Invalidate the current session and clear tokens
GET	/api/auth/refresh	Issue a new access token using a valid refresh token
GET	/api/auth/validate	Verify whether the current access token is still valid
GET	/api/auth/validate/email	Check if an email address is already registered
GET	/api/auth/me	Return the authenticated user's basic profile data
GET	/api/auth/me/settings	Return the authenticated user's app and notification settings
PATCH	/api/auth/account/modify	Update account details such as name, nickname, or avatar
DELETE	/api/auth/account/delete	Permanently delete the authenticated user's account
PATCH	/api/auth/change-password	Change password for an authenticated user
POST	/api/auth/forget-password	Initiate password reset flow, sends a reset link by email
POST	/api/auth/reset-password	Complete password reset using the token received via email
GET	/api/auth/google-login	Redirect user to Google OAuth2 login page (FUTURE DEVELOPMENT)
GET	/api/auth/google-callback	Handle OAuth2 callback from Google, establish session (FUTURE DEVELOPMENT)
Profile		
GET	/api/identity/profile/{Id}	Return public profile of any user (student: basic info; teacher: courses, ratings, qualifications)
Cities — Location Data		
GET	/api/cities/all	Return all available cities
GET	/api/cities/search	Search cities by name keyword
GET	/api/cities/postal/search	Search postal codes by keyword
GET	/api/cities/search/city_by_postal	Look up city name by postal code
GET	/api/cities/search/postal_by_city	Look up postal code by city name
CourseBase — Course Management		
GET	/api/courses	Return courses with optional filters (teacher, pagination)
GET	/api/courses/all	Return all courses without filters (admin/internal use)
POST	/api/courses/add	Create a new course (teacher only)
GET	/api/courses/{courseId}	Return full details of a specific course
PATCH	/api/courses/{courseId}	Update metadata of an existing course (teacher only)
CourseMetadata — Taxonomies & Tags		
GET	/api/courses/metadata/domains	Return all available course domains/categories
POST	/api/courses/metadata/add/domains	Add a new course domain (admin only)
GET	/api/courses/metadata/levels	Return all available course difficulty levels
POST	/api/courses/metadata/add/levels	Add a new difficulty level (admin only)
GET	/api/courses/metadata/tags	Return all available course tags

Method	Endpoint	Description
GET	/api/courses/metadata/tags/{keyword}	Search tags by keyword (autocomplete, triggers after 2 chars)
POST	/api/courses/metadata/add/tags	Add a new course tag (admin only)
Tutoring — One-on-One Courses		
POST	/api/tutoring/enrollment	Student applies to enroll in a tutoring course
PATCH	/api/tutoring/enrollment/react	Teacher accepts or rejects a pending enrollment request
GET	/api/tutoring/enrollment/teacher	Return all enrollment requests received by the teacher
GET	/api/tutoring/{studentId}/walls	Return all tutoring walls for a specific student
GET	/api/tutoring/{wallId}/posts	Return all posts on a specific tutoring wall
GET	/api/tutoring/{wallId}/{postId}	Return a single post with its comments and attachments
POST	/api/tutoring/wall/post	Teacher creates a new post on a tutoring wall
POST	/api/tutoring/wall/post/comment	Add a comment to a wall post (student or teacher)
POST	/api/tutoring/wall/post/handin	Student submits an assignment hand-in on a wall post
Scheduling — Lessons & Availability		
GET	/api/scheduling/get-events	Return upcoming scheduled events for the authenticated user
POST	/api/scheduling/book-event	Book a lesson slot with a teacher
DELETE	/api/scheduling/events/{eventId}	Cancel a scheduled event
GET	/api/scheduling/week-free-timeblocks	Return the teacher's configured weekly availability blocks
POST	/api/scheduling/week-free-timeblocks	Teacher sets a new weekly availability time block
DELETE	/api/scheduling/week-free-timeblocks/{blockId}	Teacher removes an availability time block
GET	/api/scheduling/{teacherId}/free-days	Return days with available slots for a given teacher
GET	/api/scheduling/{teacherId}/free-times	Return available time slots on a specific day for a teacher
Communication — Chats, Notifications & Reviews		
GET	/api/communication/chats	Return all chat conversations for the authenticated user
GET	/api/communication/chats/{chatId}	Return message history for a specific chat
POST	/api/communication/chats/{chatId}	Send a new message in a specific chat
GET	/api/communication/notifications	Return unread notifications for the authenticated user
POST	/api/communication/write_review	Student submits a review and rating for a completed course
FileManager — File Upload & Serving		
POST	/api/files/profile-picture	Upload or replace the user's profile picture
POST	/api/files/course-icon-picture/{courseId}	Upload or replace the icon image for a course
POST	/api/files/course-banner-picture/{courseId}	Upload or replace the banner image for a course
GET	/files/{storagePath}	Serve a stored file (image, PDF, video) by its storage path
LookUp — Reference Data		
GET	/api/lookups/currencies	Return all supported currencies
POST	/api/lookups/currencies/add	Add a new supported currency (admin only)
GET	/api/lookups/languages	Return all supported interface/course languages
POST	/api/lookups/languages/add	Add a new supported language (admin only)
Preference — Course & Learning Preferences		
GET	/api/preferences/all	Return all defined preferences

Method	Endpoint	Description
GET	/api/preferences/all/group	Return all preference groups
GET	/api/preferences/all/by_group	Return preferences organised by their group
GET	/api/preferences/all/user	Return preferences selected by the authenticated user
POST	/api/preferences/add/new	Create a new preference option (admin only)
POST	/api/preferences/add/new/group	Create a new preference group (admin only)
POST	/api/preferences/add/to_user	Assign selected preferences to the authenticated user's profile
Payment		
POST	/api/payment	Submit a payment for a course or lesson
PATCH	/api/payment/react	Update payment status by teacher whether the payment was completed
Pages — Composite Page Data (BFF Aggregation Endpoints)		
GET	/api/pages/student/homepage	Student home: active courses, upcoming events, recommendations, notifications
GET	/api/pages/student/my-courses	Student's full list of active and completed course enrolments
GET	/api/pages/student/walls/{wallId}	Bundled tutoring wall data for the student view
GET	/api/pages/student/invoices	Student's payment history and outstanding invoices
GET	/api/pages/teacher/homepage	Teacher home: active courses and upcoming lesson overview
GET	/api/pages/teacher/my-courses	Teacher's full list of created courses
GET	/api/pages/teacher/my-students	All students enrolled across any of the teacher's courses
GET	/api/pages/teacher/courses/{courseId}/students	Students enrolled in a specific course
GET	/api/pages/teacher/walls/{wallId}	Bundled tutoring wall data for the teacher view
GET	/api/pages/teacher/invoices	Teacher's earnings history and pending payouts
GET	/api/pages/teacher/course-creator	Reference data for the course creation form (languages, currencies, domains, levels)
GET	/api/pages/course-explorer	Paginated, filterable public course listing for the course explorer page

Használt források

Frontend

1. Next.js dokumentáció – https://nextjs.org/docs
2. React dokumentáció – https://react.dev
3. TailwindCSS dokumentáció – https://tailwindcss.com/docs
4. ShadCN/UI komponenskönyvtár – https://ui.shadcn.com

Backend

6. ASP.NET Core dokumentáció – https://learn.microsoft.com/en-us/aspnet/core
7. ASP.NET Core Identity – https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity
8. Entity Framework Core dokumentáció – https://learn.microsoft.com/en-us/ef/core
9. JWT Bearer Authentication – https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn

Adatbázis

10. PostgreSQL dokumentáció – https://www.postgresql.org/docs
11. EF Core – PostgreSQL provider (Npgsql) – https://www.npgsql.org/efcore

Infrastruktúra

12. Docker dokumentáció – https://docs.docker.com
13. Docker Compose – https://docs.docker.com/compose

Tervezési eszközök és egyéb

14. C4 modell – architekturális diagram módszertan – https://c4model.com
15. Figma – UI tervezőeszköz – https://www.figma.com
16. Excalidraw – UI sketch tervezőeszköz – https://excalidraw.com/